omni

Turn scattered reporting into shared data models.

Your self-guided introduction to the
Data Model Accelerator

Plan the whole migration.
Deliver complete business domains.

Learn what the tool does, explore a fictional Billing domain, review a delivery, and install it in your approved assistant.

DRAFT — requires explicit SE/AE review before sending to customer.

Documentation checked October 28, 2026
Workflow baseline fcec34f · Six lessons · Synthetic examples

Choose your starting point

Read the lessons in order, or jump to the task in front of you. You can learn and review a delivery before installing anything.

1. Understand the accelerator

2. Follow a migration

3. Explore a complete domain

4. Review a delivery

5. Install in your approved assistant

6. Launch your migration

Have a delivery ZIP? Start with lesson 4. Already installed? Verify the selected copy in lesson 5, then launch in lesson 6.

These six lessons match the editable Markdown guide in the repository. Underlined references open source files or official documentation. The optional commerce practice is a separate, smaller review exercise.

Illustrations explain the workflow; they are not UI screenshots. Synthetic files and local checks do not establish native-host, warehouse or business acceptance.

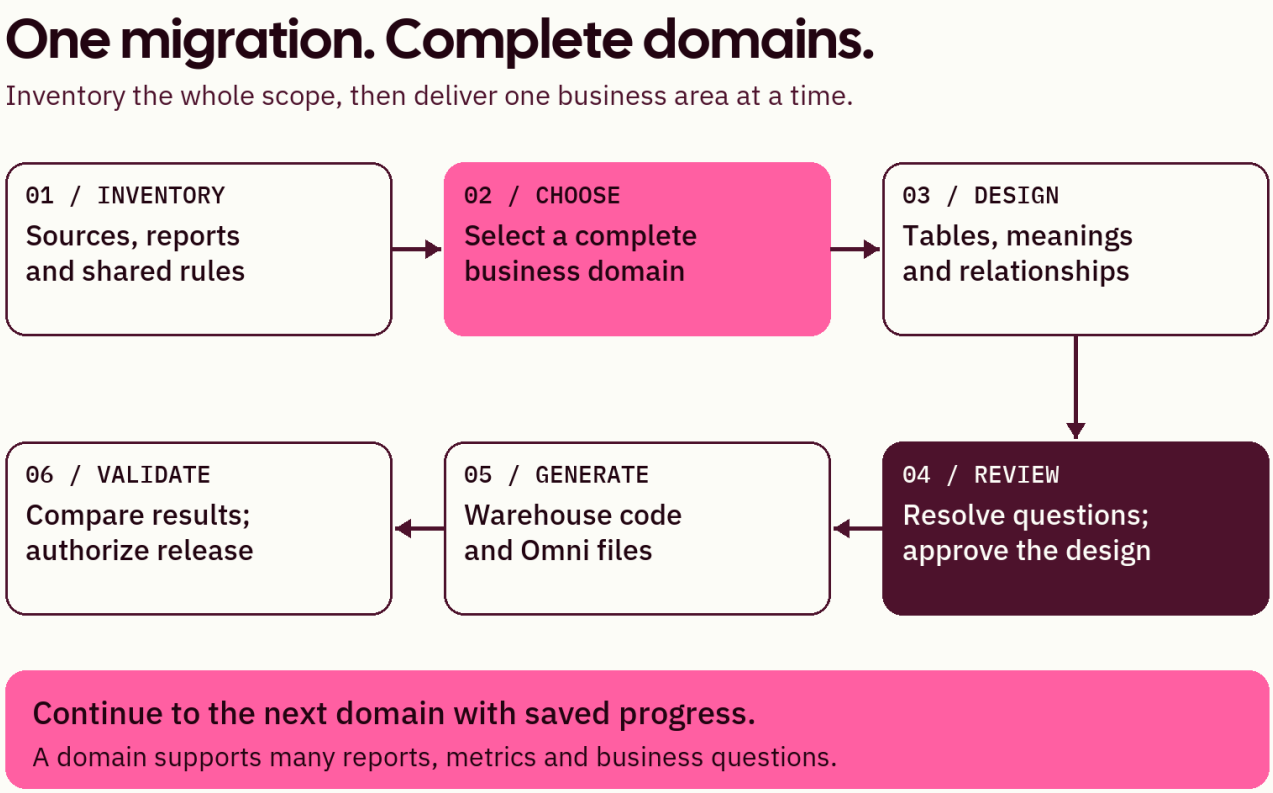
1. Understand the accelerator

Your goal: explain what the tool produces and where people make decisions.

The Data Model Accelerator helps turn scattered reporting logic into a shared, documented data model: tables with clear meanings, keys and relationships. It supports migration planning, refactoring existing models, and designing new domains from raw data.

It runs through your team's AI coding assistant. The repository supplies skill instructions and supporting tools; it is not a separate website you sign into. Exact capabilities depend on the supplied inputs, the assistant's tools and the chosen destination.

How the work moves



Teaching illustration. A domain supports many reports, metrics and business questions.

The **coordinator** organizes the engagement, asks missing questions and retains progress. **Specialists** interpret sources, design tables, prepare selected code and review evidence. They receive focused context. Some assistants use separate subagents; others perform disclosed sequential specialist tasks.

You do not need to manage those roles yourself. Your team provides business meaning, verifies data and authorizes releases. In a small company, one person may perform several responsibilities; technical validation still needs appropriate access and skills.

What it helps you do

Situation	Useful outcome
Reports repeat logic and disagree	Find shared rules, expose conflicts and propose reusable models.
Existing models need rebuilding	Design a better structure and explain what replaces or reuses old work.
Reporting is moving to Omni	Prepare warehouse models, Omni semantic files and migration mappings.
A business area has never been modeled	Propose a domain from source evidence and business definitions.

What you receive

Select the outputs you need: diagrams, a dictionary, layer documentation, warehouse code, Omni files, historical mappings, and optional migration plans or ticket templates. A guided `START_HERE.html` brings the selected deliverables together.

An Omni topic is a curated starting point for exploring related data. It is not a rebuilt dashboard. Dashboard migration continues through an installed Omni Artifact Accelerator or an explicit engineering handoff.

Current release: suitable for supervised pilots. Syntax checks and generated files do not prove business definitions, matching results or effective access controls. Provisional code remains available when requested, with unresolved assumptions visible.

You're ready when

You can distinguish a model proposal, generated code and a validated result, and identify who will review business meaning and technical behavior.

2. Follow a migration

Your goal: understand the complete migration while choosing a manageable delivery sequence.

Imagine a fictional subscription business moving scattered reporting logic into shared warehouse models and Omni. This wider company scenario is illustrative. The Billing artifacts in the next lesson come from the maintained synthetic Hex example. Sales and Product Usage are teaching domains, not completed implementations.

Inventory the whole agreed scope

Supply repositories, report/workbook exports, a warehouse catalogue and business rules. Identify the old implementation separately from the future repository. Existing work in the future repository is a reuse candidate, not automatically correct.

Account for every supplied in-scope asset, including unread, unsupported, excluded and unresolved items. A partial export must not appear complete. The agent proposes domains from business processes, relationships and shared rules across reports.

Candidate domain	Example responsibilities	Shared dependency	Teaching status
Sales	Accounts, opportunities and conversion	Customer identity	Proposed; no code here
Billing	Invoices, payments, adjustments and history	Customer identity and history	Worked synthetic example
Product Usage	Events, adoption and engagement	Customer identity	Proposed; no code here

Reports are requirements to satisfy. They do not require one new table or topic each. Shared customer definitions should not be independently reinvented for each domain.

Choose the first domain

Review the boundaries and dependencies, then select a domain. Here the team chooses Billing because it has usable evidence and a reviewer who understands the definitions. This is a teaching choice, not a universal priority rule.

The agent designs the related tables, exposes unresolved decisions and iterates with you. After scoped design approval it generates selected warehouse and semantic files. You can explicitly request provisional code before every answer is available.

Keep the whole migration visible

Track separately	What it tells you
Inventory and mappings	Which old assets have destinations, exceptions or unresolved gaps
Domain progress	What is queued, active, paused, completed or stale
Code and semantic delivery	Which requested files actually exist
Validation and acceptance	What was checked, by whom and against which version
Deployment and retirement	What was authorized and confirmed in the real environment

An asset can be accounted for without being migrated. Proposed retirement or deferral needs a decision; it must not quietly reduce migration coverage. Domain completion does not establish dashboard acceptance or production deployment.

Keep the same saved engagement for the next domain. Reuse unchanged extraction and reviewed shared models. Revisit affected dependencies when a shared definition changes.

You're ready when

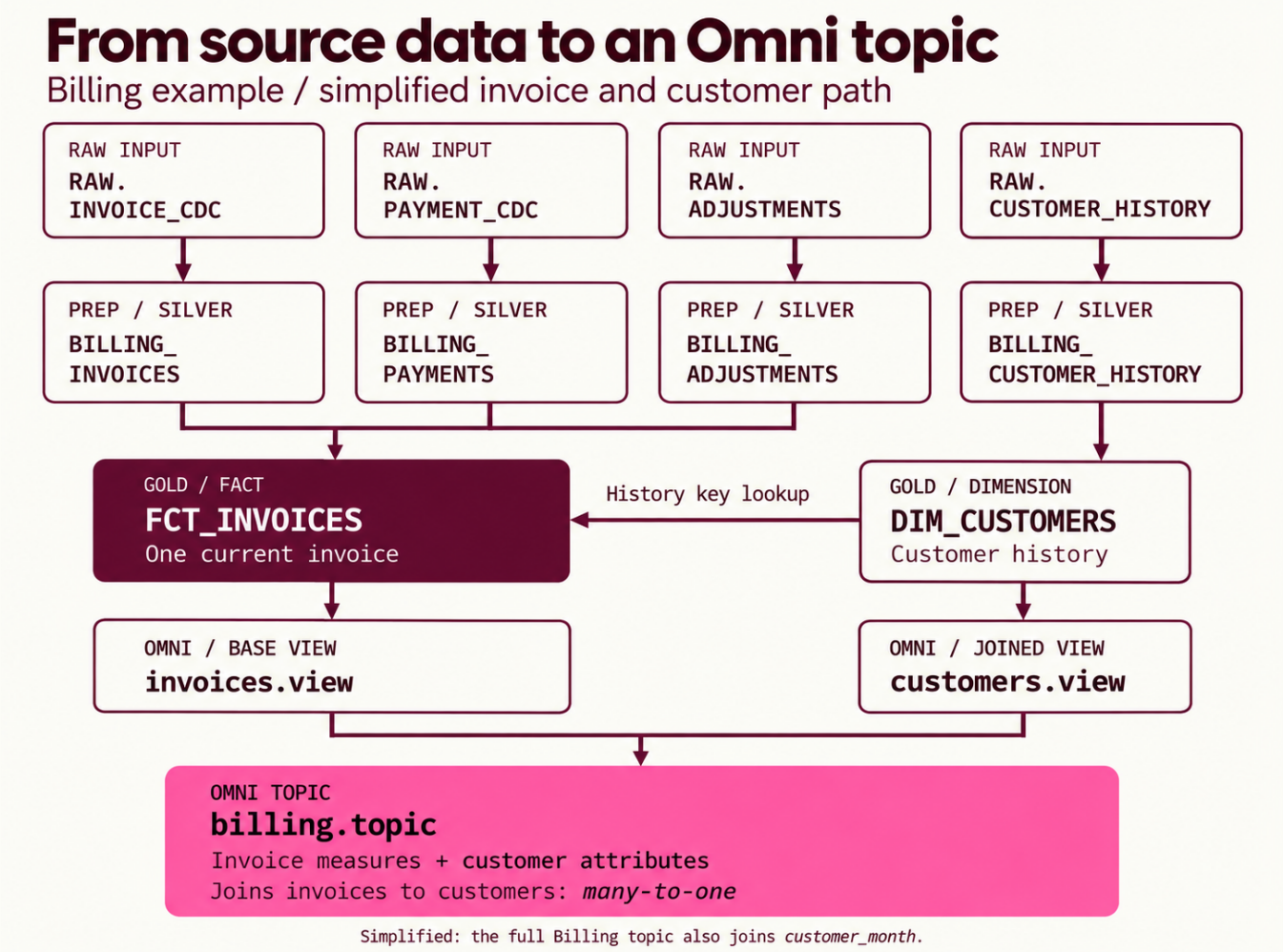
You can distinguish migration-wide scope from the active domain, identify a shared dependency, and explain why generating Billing code does not finish the company's migration.

3. Explore a complete domain

Your goal: see how one domain supports several reports and definitions.

The maintained Billing example contains four raw contracts, four preparation models and three gold models: **11 scoped warehouse objects**. It includes three Omni views, two topics and source-to-semantic mappings. These are synthetic candidates, not live deployment evidence.

Follow warehouse models into an Omni topic



Simplified teaching illustration based on the example files. The full Billing topic also joins `customer_month`; that branch is omitted here for clarity. Declared relationships still require data validation.

Raw inputs become prepared data, then reusable gold tables. The fact uses customer history to resolve the invoice-date customer key. Each gold table binds to its own Omni view. **The topic brings the views together through a defined relationship**: invoice measures can be explored alongside customer attributes. It defines how to query the models; it does not create another warehouse table.

Understand what one row means

Gold model	Grain: what one row represents	Purpose
`DIM_CUSTOMERS`	One customer history period within an organization	Retain classification at the relevant time
`FCT_INVOICES`	One current invoice within an organization	Combine invoice, payment and signed adjustment amounts
`FCT_CUSTOMER_MONTH`	One customer's invoice activity per month and organization	Monthly activity and an approximate starting month

The sample calls an organization a **tenant**: its records stay separate. If a customer changes segment in February, January invoices retain the January segment. The shared models support revenue, payment, balance, executive and retention-related reporting.

The example retains drafts in shared tables and applies posted-invoice restrictions in selected semantic measures. It keeps two active-customer meanings distinct. First-paid-month approximates a starting month from invoice months and current paid amounts; it does not record the first payment date.

Follow the artifacts

Inspect these directly in GitHub without running code:

- Layer documentation and dictionary: fields, dependencies and open decisions.
- Invoice model SQL: sums payments before joining invoices, with tenant-scoped identities.
- Omni invoices view: physical binding, fields and measures.
- Billing topic, retention topic and relationships: related exploration paths.
- Semantic mapping: revenue, retention and executive source outputs mapped to definitions.

Find each file's role; you do not need to understand every line. Fixture names and rules are not defaults for your company. Retain your selected naming policy and reviewed definitions.

Try one review decision

One report defines active customers by positive invoiced value; another uses positive payment value. Should the accelerator merge them automatically?

Check your answer: no. Retain clearly named alternatives and ask the business owner to resolve the disagreement. Code can remain provisional. Matching one report does not make its definition authoritative.

You're ready when

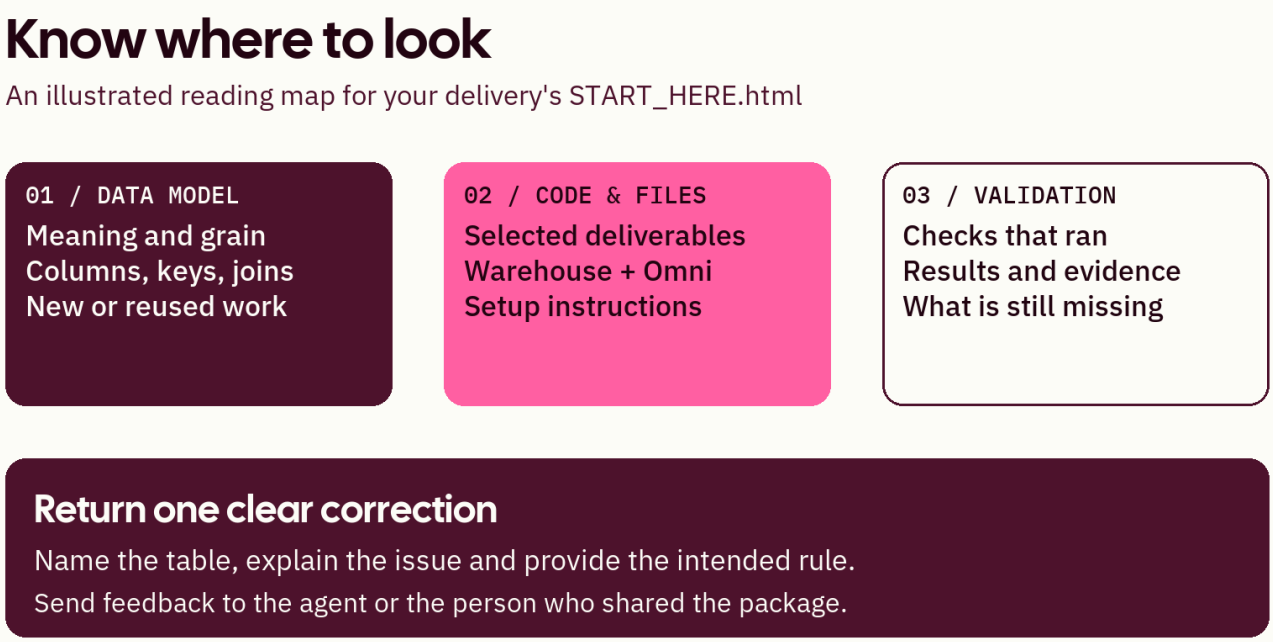
You can explain how these models serve several reports, locate warehouse and Omni files, and identify a decision requiring review. Native dbt/Snowflake execution, Omni validation, dashboard behavior and effective access remain separate checks.

4. Review a delivery

Your goal: find information and return a specific correction.

Download and extract the delivery ZIP, then open `START_HERE.html` from the extracted folder. Keep the folders together. It works offline; GitHub's HTML preview does not run it. You do not need the accelerator installed to review a package.

Visit these three areas



Illustrated navigation map, not a screenshot. Controls vary with package layout, selected files and available evidence.

1. **Data model:** in a simple package, choose a domain and gold output. Trace sources, then select a table for purpose, grain, columns, keys, new/reused status and linked code. A classic package uses the detailed ERD.

2. **Code & files:** find the selected narrative, implementation and engineering instructions. Reviewer packages exclude code; request an engineering package when needed.

3. **Validation:** inspect what actually ran, what was compared and what remains unverified. File integrity is not warehouse or business acceptance.

Inspect one table to learn the controls, then review the domain's full scope. Your starting table does not limit the model to one question. Display filters do not approve scope changes or shrink the full delivery.

Find files without the page

Default simple folder	Contents
`01_Input_Sources/`	Input references and explicitly included permitted files
`02_Input_Context_Files/`	Included business rules and context
`03_Design_Documents/`	Diagrams, dictionary, layer guidance and narrative
`04_Code_Generation/`	Selected code, setup guidance and validation files
`05_Mappings/`	Old-to-new comparisons and migration planning files

Folder guides explain omissions. Original repositories are not automatically copied. Classic packages use `01_Model/`, `02_Implementation/` and `03_Validation/`; follow the package README.

Return actionable feedback

Name the table, identify the issue and provide the rule or evidence needed. Example: “In Billing, retain invoice-date customer classification. Do not replace it with current classification. Show the affected models and reports.”

Reply in chat, or use **Prepare review feedback**, then **Download feedback**, then return that file to the agent. Send the downloaded feedback file—or your written comments—to the person who sent the package. The offline page does not submit feedback automatically.

Know what you are approving

State	Meaning
Draft or provisional	Reviewable output with unresolved assumptions or checks
Local checks passed	Stated local checks passed for identified files
Native validation recorded	Specified checks ran in the named warehouse or Omni environment
Business approval recorded	A person approved the exact definitions and scope
Deployment confirmed	An authorized deployment returned a confirmed result

Answering a question is not automatically design approval. Design approval does not authorize deployment. Keep failures, unsupported behavior and missing evidence visible.

You're ready when

You can find a definition, locate requested code or explain its absence, identify a missing check and return a correction. Optional: practice with a small review package.

5. Install in your approved assistant

Outcome: your assistant can find the complete accelerator and identify its active copy. Installation needs no warehouse credentials.

Official setup documentation checked October 28, 2026; native installations were not exercised.

1. Check what you already have

Use your organization's approved assistant. Ask your support contact if one is not available.

Have a technical partner confirm the maintained runtime: **Python 3.12 on macOS or Linux (POSIX)**, helper dependencies and file/execution permissions. Windows needs a separately qualified Linux environment, such as WSL or a remote environment.

Cloud skill discovery does not prove helper execution or file access. Keep those checks separate.

2. Prepare a separate learning folder

Use a reviewed repository copy with its revision (version identifier) recorded. Open your assistant in a new `dma-learning` folder **outside customer repositories**.

The required folder is `skills/data-model-accelerator` from the reviewed copy. Keep its entire contents, including `SKILL.md`, scripts, references, and supporting files. Copying only the Markdown instructions is incomplete. Add optional stage skill folders only when needed, beside the main folder under the same skills parent.

Paths below are relative to your learning folder, except Databricks workspace paths. Compare existing installations before replacing anything.

3. Follow one host path

Codex

Codex documents `$skill-installer` for repository skills. If available, ask it to install the **complete reviewed folder at the recorded revision** into `.agents/skills/data-model-accelerator/`; have it identify its destination first. If that installer cannot target your learning folder, manually copy the complete folder there instead. Open that workspace, then select the skill through the desktop skill picker or `/skills` in the CLI. Restart if it is missing. Official Codex skills guide

Claude Code

Copy the complete main folder to `.claude/skills/data-model-accelerator/`. Start a local Claude Code session in your learning folder and invoke `/data-model-accelerator`. If the skills directory was created after the session started, use `/reload-skills`. This is the local Claude Code path; cloud sessions have separate loading rules. Official Claude Code skills guide

Gemini CLI

Use its native installer from your learning folder. Ask your technical partner to substitute the reviewed folder's actual path:

```
gemini skills install /reviewed-copy/skills/data-model-accelerator --scope workspace
```

Keep the installation confirmation enabled. Then use **`/skills reload`** and **`/skills list`**. Ask Gemini to activate **`data-model-accelerator`**, and check the folder shown in its activation prompt. The manual fallback is the complete folder at **`.gemini/skills/data-model-accelerator/`**. Official Gemini skills guide

Snowflake Cortex Code / CoCo Desktop

For Desktop, copy the complete folder to **`.snowflake/cortex/skills/data-model-accelerator/`** and open the learning workspace. Alternatively, use **Agent Settings > Skills > Local Skills > +**, selecting a parent folder containing your reviewed skill folders. This user-level registration is shared with the CLI. Confirm the skill appears and look for "Using skill" when invoked. Official CoCo Desktop skills guide

For CLI-only project installation, its documentation specifies **`.cortex/skills/data-model-accelerator/`**; check **`/skill list`**, then ask it to use the skill. Official CoCo CLI guide

Cortex Agents is a separate cloud route with different supporting-file restrictions. Do not flatten this package or assume a stage upload qualifies its nested helpers for execution. Cortex Agents limitations

Databricks Genie Code

Have your technical partner place the complete reviewed folders in a personal workspace learning area, for example **`/Users/{username}/dma-learning/skills/`**. In Genie Code **Settings**, enter that parent workspace path and choose **Add folder**. Alternatively, copy the complete main folder to **`/Users/{username}/.assistant/skills/data-model-accelerator/`**. These are Databricks workspace paths, not your laptop's folders. Start a new chat and **`@`** mention the skill. Helper execution still needs separate qualification. Official Genie Code skills guide

4. Confirm the selected copy

Send this request after selecting the skill:

> Use data-model-accelerator. Identify the exact SKILL.md path you loaded. Report the reviewed source revision, or say if it is unavailable. Read its installation reference and identify a referenced helper file you can access. Compare this installed copy with the reviewed copy using check_installation.py. Report file identity and runtime availability separately. Do not change global settings or overwrite another installation.

The preflight compares file identity; it **does not prove the host loaded the skill**. Check the host's invocation indication and the assistant's specific path/reference response too. A copied folder may need the source revision recorded separately.

5. Resolve gaps and continue

If the skill is absent, check folder nesting, exact **`SKILL.md`** spelling, the open workspace, and whether the skill is disabled. If two copies appear, identify the selected path before continuing. Missing Python, dependencies, permissions, or inaccessible references means setup needs your technical partner's help; do not bypass controls.

Checkpoint: you can name the selected copy, its reviewed source, and any remaining execution gap. Continue to launch your first task; synthetic practice is optional.

6. Launch your migration

Your goal: start a saved migration with clear scope and a complete first domain.

Prepare what you have

- Migration outcome and approved repositories or exports in scope.
- Which implementation is legacy and which repository is the destination.
- Reports and dashboards that must be satisfied or reviewed for retirement.
- Available warehouse tables/columns, business rules, naming policy and known errors.
- Preferred warehouse, transformation tool and Omni scope; unknown is acceptable.
- People who can review definitions and validate implementation; roles may overlap.
- Desired design, code, semantic and mapping outputs; optional Excel/Word or Google migration plans.

Use approved locations and sanitized evidence. Keep credentials and sensitive customer records out of prompts. The agent should ask targeted questions and retain unresolved decisions.

Copy this kickoff prompt

Replace the bracketed fields and paste into your configured coding assistant.

```
Use the Data Model Accelerator. Confirm the active installation.
Help us migrate or refactor the whole agreed scope.

Outcome: [what is changing and why].
Legacy inputs: [approved repository/export locations].
Future repository and reusable models: [location or unknown].
Reports and dashboards to satisfy: [inventory or unknown].
Warehouse catalogue: [location or unknown].
Tools and warehouse: [choices or unknown].
Omni and dashboard scope: [choices or unknown].
Business rules, known errors and naming: [details or unknown].
Reviewers: [roles or unknown].
Save outside the input repositories: [new engagement folder].

Keep sources read-only. Inventory the agreed scope, propose
domains and shared dependencies, and recommend an order.
Ask which domain we want first. Design that complete domain
for its related reports, metrics and business questions.
Use inventories as requirements, not one model per report.
Show reused, new and replaced models and unresolved mappings.

Provide diagrams, dictionary, layer guidance and decisions
for review before implementation. Ask which code, semantic
and migration-plan outputs we want. Preserve naming choices.
Keep requested provisional code available with missing context.
Retain progress and provide one current review package.
Do not deploy, create live tickets or retire sources at intake.
```

What should happen next

Expect coverage and input summaries, missing-context questions, domain choices, dependencies and a saved location. Select a domain, then review its entities and rules. Incomplete inventory remains a visible gap.

When ready, say: “I approve the displayed design for [domain/version]. Generate the selected warehouse and Omni candidates. This approves candidate generation, not execution or deployment.” Include Omni only if selected.

For earlier code, say: “Generate provisional files for [domain]. Label unresolved bindings and business rules. Do not infer approval.” Design documents alone do not complete a code request.

Continue toward the full migration

Choose the next domain after the current agreed work. Resume the saved engagement and reuse unchanged inputs and models. Shared-definition changes may require rechecking dependent domains.

For reporting migration, use the optional Omni Artifact Accelerator handoff or an explicit engineering route. Topics alone do not rebuild dashboards. Require independent comparisons and access checks; separately authorize execution, release and retirement. Retain cutover and recovery responsibilities.

Track what is inventoried, mapped, generated, validated, accepted, deployed, deferred and unresolved. Mappings and documents do not prove parity. Deferred work must remain visible instead of silently reducing scope.

You're ready when

You have a saved location, an agreed first domain, review responsibilities or visible ownership gaps, and expected outputs. Begin the engagement without treating the learning example as production evidence.